Basilisk 101: Computational Fluid Dynamics with Basilisk C

Heat Conduction Simulation: From Vanilla C to Basilisk C

Vatsal Sanjay

Web version

GitHub repository

Click here

TL;DR:

Heat Conduction Simulation Tutorial This tutorial walks through implementing heat conduction simulations, starting with basic vanilla C implementations and progressing to more powerful Basilisk C versions. We build from 1D steady-state problems to 2D, annular, and axisymmetric geometries, demonstrating how Basilisk's domain-specific language dramatically simplifies code while adding powerful features like adaptive mesh refinement and embedded boundaries. The tutorial includes eight progressive simulation cases, showing how the same physics can be implemented with increasingly sophisticated numerical methods, and concludes with custom post-processing tools for data extraction and visualization. Basilisk ultimately reduces hundreds of lines of vanilla C code to concise, readable implementations that handle complex geometries with minimal effort.

Introduction

This guide walks you through implementing heat conduction simulations using both vanilla C and the Basilisk framework. We'll progress from simple steady-state problems to more complex transient simulations in multiple dimensions, highlighting the advantages of Basilisk for computational fluid dynamics (CFD) and heat transfer applications.

Learning Objectives

- Implement basic heat conduction solvers in vanilla C
- Transition to equivalent implementations in Basilisk C
- Understand the advantages of Basilisk's domain-specific language
- Develop simulations with increasing complexity (1D → 2D → axisymmetric)
- Appreciate how Basilisk simplifies numerical methods implementation
- Learn to analyze and visualize simulation results with post-processing tools

Throughout this tutorial, we'll follow a logical progression across eight core simulation cases:

1. Steady-state heat conduction in vanilla C (`1-conduction-simple.c`)
2. Transient heat conduction in vanilla C (`1-conduction-transient.c`)
3. Steady-state heat conduction in Basilisk (`1-conduction-simple-basilisk.c`)
4. Transient heat conduction in Basilisk (fill-in exercise) (`1-conduction-transient-basilisk.c`)
5. Enhanced transient solution with diffusion module (`1-conduction-transient-basilisk-withHeaders.c`)

6. 2D heat conduction (`1-conduction-2D.c`)
7. Heat conduction in an annulus (`1-conduction-2D-annulus.c`)
8. Axisymmetric heat conduction (`1-conduction-Axi.c`)

After completing the simulations, we'll also cover how to post-process and visualize the results using custom tools.

Part 1: Heat Conduction in Vanilla C

1.1 Steady-State Heat Conduction (1-conduction-simple.c)

We'll begin with a fundamental 1D steady-state heat conduction problem in vanilla C.

Problem Statement Solve the steady-state heat equation:

$$\frac{d^2T}{dx^2} = 0 \tag{1}$$

Subject to Dirichlet boundary conditions:

- $T(0) = 0$
- $T(1) = 1$

The analytical solution is the linear profile $T(x) = x$.

Let's examine the key elements of this implementation:

```c
// Domain discretization
#define N        11         // Number of grid points
#define MAX_ITER 10000      // Maximum number of iterations
#define TOL      1e-10      // Convergence tolerance

// Core iterative solver using Gauss-Seidel method
double update_interior_points(double t_current[N], double t_new[N]) {
  int i;
  double error = 0.0;
  double diff;

  for (i = 1; i < N - 1; i++) {
    // Central difference scheme
    t_new[i] = 0.5 * (t_current[i - 1] + t_current[i + 1]);

    // Track maximum error for convergence check
    diff = fabs(t_new[i] - t_current[i]);
    if (diff > error) {
      error = diff;
    }
  }

  return error;
}
```

This implementation uses the Gauss-Seidel iterative method with a central difference approximation for the second derivative:

$$T_i \approx \frac{T_{i-1} + T_{i+1}}{2} \tag{2}$$

Key Implementation Details

- Arrays for storing current and new temperature values
- Explicit boundary condition application
- Manual iteration until convergence
- Manual memory management
- Explicit file I/O for results

1.2 Transient Heat Conduction (1-conduction-transient.c)

Next, we'll look at a 1D transient heat conduction problem in vanilla C.

Problem Statement Solve the transient heat equation:

$$\frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial x^2} \tag{3}$$

With no-flux boundary conditions and an initial condition approximating a Dirac delta function.
The analytical solution is a Gaussian profile that spreads over time:

$$T(x,t) = \frac{1}{2\sqrt{\pi t}} e^{-x^2/4t} \tag{4}$$

Key aspects of this implementation:

```c
// Compute heat fluxes at cell interfaces
void compute_fluxes(double *temperature, double *flux, double dx) {
  flux[0] = 0.0; // Left boundary (no flux)
  flux[N] = 0.0; // Right boundary (no flux)

  for (int i = 0; i < N - 1; i++) {
    flux[i+1] = - (temperature[i+1] - temperature[i]) / dx;
  }
}

// Update temperature field based on fluxes
void update_temperature(double *t_current, double *t_new, double *flux,
                        double dx, double dt) {
  for (int i = 0; i < N; i++) {
    double dq;

    if (i == 0) {
      // Leftmost cell: flux difference is q[0.5] - q[-0.5], but q[-0.5]=0
      dq = flux[1];
    } else if (i == N-1) {
      // Rightmost cell: flux difference is q[N-0.5] - q[N-1.5], but q[N+0.5]=0
      dq = - flux[N-1];
    } else {
      dq = flux[i+1] - flux[i];
    }
```

```
    t_new[i] = t_current[i] - (dt/dx) * dq;
  }
}
```

Key Implementation Details

- Explicit time-stepping scheme (forward Euler)
- Manual computation of fluxes at cell interfaces
- Explicit handling of boundary conditions
- Stability condition for time step ($dt \leq dx^2/2$)
- Manual memory management and array allocation
- Explicit file I/O for snapshots and results

Exercise 1: Modify the Vanilla C Code

Suggested Tasks

1. Modify `1-conduction-simple.c` to solve a 2D steady-state problem
   - Add a second spatial dimension to the grid
   - Update the Gauss-Seidel iteration for 2D
   - Modify the boundary conditions for all four edges
2. Modify `1-conduction-transient.c` to use a different initial condition
   - Replace the Dirac delta with a Gaussian profile: $T(x,0) = e^{-x^2/\sigma^2}$
   - Compare the numerical and analytical solutions at different times

Part 2: Introduction to Basilisk C

2.1 Steady-State Heat Conduction in Basilisk (1-conduction-simple-basilisk.c)

Now let's examine the equivalent steady-state problem implemented in Basilisk C.

```
#include "grid/cartesian1D.h"
#include "run.h"

// Declare scalar field for temperature
scalar T[];
T[left] = dirichlet(0.0);
T[right] = dirichlet(1.0);

int main() {
  // Domain setup
  L0 = 1.0;      // Domain length
  X0 = 0.0;      // Left boundary
  N = 200;       // Number of cells

  // Set the timestep based on stability criterion
  DT = (L0/N)*(L0/N)/2;

  // Create output directory
  char comm[80];
```

```
  sprintf (comm, "mkdir -p intermediate");
  system(comm);

  // Run simulation
  run();
}

// Time integration using explicit finite volume method
event marching (i++) {
  foreach() {
    // Proper heat equation time stepping
    T[] += DT*(T[1] - 2*T[] + T[-1])/(Delta*Delta);
  }
}
```

Basilisk Advantages

- Simplified domain and grid setup
- Declarative boundary conditions
- Event-based programming model
- Intuitive stencil operations with `T[1]` and `T[-1]`
- Automatic handling of grid traversal with `foreach()`
- Significantly reduced code length with the same functionality

2.2 Understanding Basilisk's Event System

Basilisk uses an event-based programming model that separates concerns:

- `event init (t = 0)`: Initialization at t=0
- `event marching/integration (i++)`: Time integration at each step
- `event end (t = end)`: Final operations at the end of the simulation

Events can also be scheduled at specific times or intervals:

```
// Execute every 0.1 time units
event writingFiles (t = 0.0; t += 0.1; t < 1.0) {
  // Save output
}
```

Event Scheduling Syntax The event scheduling syntax in Basilisk follows this pattern:

```
event name (condition) { ... }
```

Where condition can be:

- `t = 0` - Single execution at t=0
- `i++` - Every iteration
- `t += tsnap` - At regular time intervals
- `t = end` - At the end of simulation
- Combinations like `t = 0; t += 0.1; t < 1.0` for repeated execution

Exercise 2: Understand Basilisk Code Structure

Tasks

1. Compare the structure and length of `1-conduction-simple.c` and `1-conduction-simple-basilisk.c`
   - How many lines of code are in each?
   - Which sections of code are simplified or eliminated in the Basilisk version?
2. Identify the advantages of the Basilisk implementation
   - List at least three specific advantages
   - For each advantage, explain how it simplifies code development or maintenance
3. Modify `1-conduction-simple-basilisk.c` to output the relative error
   - Add code to calculate and print the error relative to the analytical solution
   - Use the formula: $error = |T_{numerical} - T_{analytical}|/|T_{analytical}|$

Part 3: Transient Heat Conduction in Basilisk

3.1 Fill-in-the-Blanks Exercise (1-conduction-transient-basilisk.c)

Let's implement the transient heat equation in Basilisk. Here's a template with some parts for you to fill in:

```
#include "grid/cartesian1D.h"
#include "run.h"

// Declare scalar field for temperature
scalar T[];

// Simulation parameters
#define EPS 0.1  // Width of initial temperature peak
#define tmax 1.0
#define tsnap 0.1

int main() {
  // Domain setup
  L0 = 10.0;     // Domain length
  X0 = -L0/2;    // Left boundary
  N = 200;       // Number of cells

  // Set the timestep based on stability criterion (CFL condition)
  // dt = dx^2/2 for explicit scheme
  DT = (L0/N)*(L0/N)/2;

  // Create output directory
  char comm[80];
  sprintf (comm, "mkdir -p intermediate");
  system(comm);

  // Run simulation
  run();
}
```

```
/
 * Initialize temperature field
 *
 * Sets up a "Dirac delta" approximated by a thin rectangle
 * centered at x=0 with total integral = 1.
 */
event init (t = 0) {
  foreach()
    T[] = (fabs(x) < EPS) ? 1.0/EPS/2.0 : 0.0;
}

/
 * Time integration using explicit finite volume method
 */
event integration (i++) {
  // Get timestep for this iteration
  double dt = dtnext(DT);

  // Compute fluxes at the faces of the cells
  // q_i = -(T_i - T_{i-1})/Delta
  scalar q[];
  foreach()
    q[] = -(T[] - T[-1])/Delta;

  // Update temperature
  // dT/dt = -(q_{i+1} - q_i)/Delta
  // this enforces the central difference scheme
  scalar dT[];
  foreach()
    dT[] = -(q[1] - q[])/Delta;

  // Explicit Euler step
  foreach()
    T[] += dt*dT[];
}

/
 * Save snapshots at regular intervals
 */
event writingFiles (t += tsnap; t < tmax+tsnap) {
  char filename[100];
  sprintf(filename, "intermediate/snapshot-%5.4f.csv", t);

  FILE *fp = fopen(filename, "w");
  foreach() {
    fprintf(fp, "%g,%g\n", x, T[]);
  }
  fclose(fp);
}

/
 * Save final results and comparison with analytical solution
 */
event end (t = end) {
  char filename[100];
  sprintf(filename, "conduction-transient.csv");

  FILE *fp = fopen(filename, "w");
  foreach() {
    fprintf(fp, "%g,%g\n", x, T[]);
```

```
  }
  fclose(fp);
}
```

Stability Condition (K value) The stability condition for the 1D heat equation with explicit Euler time-stepping requires:

$$dt \leq \frac{dx^2}{2} \tag{5}$$

So K = 2 is the appropriate value to ensure numerical stability.

Understanding The Heat Equation Discretization The heat equation $\frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial x^2}$ can be discretized in two steps:

1. Compute heat fluxes at cell interfaces: $q_{i+1/2} = -\frac{T_{i+1} - T_i}{\Delta x}$
2. Update temperature based on flux divergence: $\frac{dT_i}{dt} = -\frac{q_{i+1/2} - q_{i-1/2}}{\Delta x}$

In Basilisk's staggered grid, `q[]` is at cell faces while `T[]` is at cell centers.

Part 4: Enhanced Transient Solution with Diffusion Module

Now let's look at an enhanced implementation using Basilisk's built-in diffusion module.

```
#include "grid/multigrid1D.h"  /* Multigrid solver is required by diffusion.h */
#include "run.h"
#include "diffusion.h"

// Declare scalar field for temperature
scalar T[];
// Boundary conditions
// The diffusion solver will use homogeneous Neumann conditions by default
T[left] = neumann(0.);
T[right] = neumann(0.);

// Simulation parameters
#define EPS 0.1  // Width of initial temperature peak
#define tmax 1.0
#define tsnap 0.1

int main() {
  // Domain setup
  L0 = 10.0;     // Domain length
  X0 = -L0/2;    // Left boundary
  N = 10000;     // Number of cells

  // We can use a larger timestep with the implicit solver
  // compared to the explicit version which requires dt = dx^2/2
  DT = 0.01;

  // Create output directory
  char comm[80];
  sprintf (comm, "mkdir -p intermediate");
  system(comm);
```

```
  // Run simulation
  run();
}

/
 * Initialize temperature field
 */
event init (t = 0) {
  foreach()
    T[] = (fabs(x) < EPS) ? 1.0/EPS/2.0 : 0.0;
}

/
 * Time integration using implicit diffusion solver
 */
event integration (i++) {
  // Get timestep for this iteration
  double dt = dtnext(DT);

  // Use the diffusion() function from diffusion.h to solve the equation
  diffusion(T, dt);
}
```

Advantages of Using diffusion.h

- Implicit time stepping allows for much larger timesteps
- Unconditional stability removes the CFL restriction
- Higher accuracy due to careful implementation of boundary conditions
- Multigrid acceleration for faster convergence
- Adaptive mesh refinement compatible
- Handles complex geometries through embedded boundaries

Part 5: Advanced Heat Conduction Problems

5.1 2D Heat Conduction (1-conduction-2D.c)

Now, let's extend our simulations to two dimensions.

Problem Statement Solve the 2D transient heat equation:

$$\frac{\partial T}{\partial t} = \nabla^2 T \tag{6}$$

With Dirichlet boundary conditions:

- $T = 1$ on the top boundary
- $T = 0$ on all other boundaries

Key aspects of the 2D implementation:

```
#include "run.h"
#include "diffusion.h"

// Declare scalar field for temperature
```

```
scalar T[];

// Boundary conditions
T[top] = dirichlet(1.);
T[bottom] = dirichlet(0.);
T[left] = dirichlet(0.);
T[right] = dirichlet(0.);

int main() {
  // Domain setup
  L0 = 10.0;      // Domain length
  X0 = -L0/2;     // Left boundary
  Y0 = -L0/2;     // Bottom boundary
  init_grid (1 << 7);  // 128×128 grid (2^7)

  // Larger timestep with implicit solver
  DT = 0.01;

  // Run simulation
  run();
}

event integration (i++) {
  double dt = dtnext(DT);
  mgd = diffusion(T, dt);
}

event adapt(i++){
  adapt_wavelet ((scalar *){T},
    (double[]){1e-4},
    10);
}
```

2D Implementation Notes

- Similar structure to 1D, but now using a 2D grid
- Uses `init_grid(1 << 7)` for a 128×128 grid
- Boundary conditions for all four sides
- Implicit diffusion solver works in any dimension
- Added adaptive mesh refinement with `adapt_wavelet()`

5.2 Heat Conduction in an Annulus (1-conduction-2D-annulus.c)

Let's look at a more complex geometry: heat conduction in an annular domain.

Annulus Problem Solve the transient heat equation in an annular domain:

$$\frac{\partial T}{\partial t} = \nabla^2 T \tag{7}$$

With Dirichlet boundary conditions:

- $T = 1$ on inner boundary (r = 1)
- $T = 0$ on outer boundary (r = 4)

```
#include "run.h"
#include "embed.h"
#include "diffusion.h"

scalar T[];

// Define inner and outer radii
#define INNER_RADIUS 1.0
#define OUTER_RADIUS 4.0

event init (t = 0) {
  // Define annular geometry using embedded boundaries
  solid (cs, fs, difference (sq(OUTER_RADIUS) - sq(x) - sq(y),
                             sq(INNER_RADIUS) - sq(x) - sq(y)));

  // Set boundary conditions
  T[embed] = dirichlet (sq(x) + sq(y) < sq(INNER_RADIUS + 0.1) ? 1.0 : 0.0);
}

event integration (i++) {
  // Get timestep for this iteration
  double dt = dtnext(DT);

  // Define diffusion coefficient field (constant = 1.0)
  face vector D[];
  foreach_face()
    D.x[] = fs.x[] > 0. ? 1.0 : 0.0; // Diffusion only in fluid cells

  // Solve the diffusion equation while respecting embedded boundaries
  mgd = diffusion (T, dt, D);
}
```

Embedded Boundaries

- `embed.h` provides tools for defining complex geometries
- `solid()` creates the embedded boundary using a level-set function
- `difference()` creates the annular region by subtracting two circles
- Boundary conditions are applied using `T[embed]`
- `fs.x[]` restricts diffusion to the fluid domain

5.3 Axisymmetric Heat Conduction (1-conduction-Axi.c)

Finally, let's look at an axisymmetric problem.

Axisymmetric Problem Solve the heat equation in cylindrical coordinates with axisymmetry:

$$\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}(r\frac{\partial T}{\partial r}) + \frac{\partial^2 T}{\partial z^2} \tag{8}$$

With boundary conditions:

- Gaussian heat flux at r=0 (left boundary)
- $T = 0$ on top and right boundaries

```c
#include "axi.h"
#include "run.h"
#include "diffusion.h"

scalar T[];

int main() {
  // Domain setup
  L0 = 10.0;     // Domain length
  X0 = -L0/2;    // Left boundary
  init_grid (1 << 7);      // Number of cells

  // Larger timestep with implicit solver
  DT = 0.01;

  // Boundary conditions
  T[top] = dirichlet(0.0);
  T[left] = neumann(exp(-y*y)); // Gaussian flux profile
  T[right] = dirichlet(0.);

  // Run simulation
  run();
}

event integration (i++) {
  double dt = dtnext(DT);

  face vector D[];
  foreach_face()
    D.x[] = fm.x[]; // Use metric terms for axisymmetric problems

  mgd = diffusion(T, dt, D);
}
```

Axisymmetric Features

- `axi.h` handles the axisymmetric formulation automatically
- In axisymmetric coordinates, `y` represents the radial direction (r)
- `fm.x[]` includes the metric terms for the axisymmetric Laplacian
- Neumann boundary with Gaussian profile at r=0 (left boundary)
- All the complexity of cylindrical coordinates is handled by Basilisk

Exercise 4: Advanced Heat Conduction Problems

Tasks

1. Modify the 2D heat conduction problem
   - Change boundary conditions to have a sinusoidal temperature on top: `T[top] = dirichlet(sin(π*x/L0))`
   - Compare the steady-state solution with the new boundary condition
2. Experiment with the annulus problem
   - Try different inner and outer radius values
   - Implement a time-dependent boundary condition on the inner boundary
3. Extend the axisymmetric problem

- Add a heat source term using `source(T, dt)`
- Visualize the results with different flux profiles

Part 6: Post-Processing and Visualization

After running simulations, we need tools to extract and visualize the results. We'll use two key tools:

1. `getData-generic-heatEq.c` - A Basilisk program to extract data from simulation snapshots
2. `Video-generic-heatEq.py` - A Python script to create visualizations

6.1 Extracting Data with getData-generic-heatEq.c

This Basilisk utility reads simulation snapshots and extracts field data for visualization:

```
// getData-generic-heatEq.c
#include "utils.h"
#include "output.h"

scalar T[];
scalar cs[] = {1};

char filename[80];
int nx, ny, len;
double xmin, ymin, xmax, ymax, Deltax, Deltay;

scalar * list = NULL;

int main(int a, char const *arguments[])
{
  // Parse command line arguments
  sprintf (filename, "%s", arguments[1]);  // Snapshot file
  xmin = atof(arguments[2]); ymin = atof(arguments[3]);  // Domain min
  xmax = atof(arguments[4]); ymax = atof(arguments[5]);  // Domain max
  ny = atoi(arguments[6]);  // Number of points in y

  // Add fields to extract
  list = list_add (list, T);
  list = list_add (list, cs);

  // Restore the snapshot
  restore (file = filename);

  // Calculate grid parameters
  FILE * fp = ferr;
  Deltay = (double)((ymax-ymin)/(ny));
  nx = (int)((xmax - xmin)/Deltay);
  Deltax = (double)((xmax-xmin)/(nx));
  len = list_len(list);

  // Allocate memory for field data
  double  field = (double ) matrix_new (nx, ny+1, len*sizeof(double));

  // Extract data through interpolation
```

```
  for (int i = 0; i < nx; i++) {
    double x = Deltax*(i+1./2) + xmin;
    for (int j = 0; j < ny; j++) {
      double y = Deltay*(j+1./2) + ymin;
      int k = 0;
      for (scalar s in list){
        field[i][len*j + k++] = interpolate (s, x, y);
      }
    }
  }

  // Output data to stderr (will be captured by Python script)
  for (int i = 0; i < nx; i++) {
    double x = Deltax*(i+1./2) + xmin;
    for (int j = 0; j < ny; j++) {
      double y = Deltay*(j+1./2) + ymin;
      fprintf (fp, "%g %g", x, y);
      int k = 0;
      for (scalar s in list){
        fprintf (fp, " %g", field[i][len*j + k++]);
      }
      fputc ('\n', fp);
    }
  }

  // Clean up
  fflush (fp);
  fclose (fp);
  matrix_free (field);
}
```

How getData-generic-heatEq.c Works

1. Reads a simulation snapshot file
2. Takes domain extents and resolution as input
3. Restores the simulation state
4. Interpolates field values onto a regular grid
5. Outputs coordinates and field values to stderr

This approach allows flexible post-processing of any Basilisk simulation.

6.2 Visualization with Video-generic-heatEq.py

The Python script processes the extracted data and creates visualizations:

```
# Example of key components from Video-generic-heatEq.py
def gettingfield(filename, zmin, zmax, rmin, rmax, nr):
    # Run the getData utility and capture its output
    exe = ["./getData-generic-heatEq", filename, str(zmin), str(rmin), str(zmax), str(rmax),
 ↪  str(nr)]
    p = sp.Popen(exe, stdout=sp.PIPE, stderr=sp.PIPE)
    stdout, stderr = p.communicate()
    temp1 = stderr.decode("utf-8")
    temp2 = temp1.split("\n")

    # Parse the output into arrays
```

```
    Rtemp, Ztemp, Ttemp, cstemp = [], [], [], []
    for n1 in range(len(temp2)):
        temp3 = temp2[n1].split(" ")
        if temp3 == ['']:
            pass
        else:
            Ztemp.append(float(temp3[0]))
            Rtemp.append(float(temp3[1]))
            Ttemp.append(float(temp3[2]))
            cstemp.append(float(temp3[3]))

    # Convert to numpy arrays and reshape
    R = np.asarray(Rtemp)
    Z = np.asarray(Ztemp)
    T = np.asarray(Ttemp)
    cs = np.asarray(cstemp)
    nz = int(len(Z)/nr)

    R.resize((nz, nr))
    Z.resize((nz, nr))
    T.resize((nz, nr))
    cs.resize((nz, nr))

    return R, Z, T, cs, nz

def process_timestep(ti, caseToProcess, folder, tsnap, GridsPerR, rmin, rmax, zmin, zmax, lw):
    # Calculate time and filenames
    t = tsnap * ti
    place = f"{caseToProcess}/intermediate/snapshot-{t:.4f}"
    name = f"{folder}/{int(t*1000):08d}.png"

    # Extract data
    nr = int(GridsPerR * rmax)
    R, Z, T, cs, nz = gettingfield(place, zmin, zmax, rmin, rmax, nr)

    # Create visualization
    fig, ax = plt.subplots()
    fig.set_size_inches(19.20, 10.80)

    # Mask regions outside the fluid domain
    T = np.ma.masked_where(cs != 1.0, T)

    # Plot temperature field
    cntrl1 = ax.imshow(T, cmap="coolwarm", interpolation='Bilinear',
                       origin='lower', extent=[-rminp, -rmaxp, zminp, zmaxp],
                       vmax=1.0, vmin=0.0)

    # Add colorbar and save
    cb1 = fig.add_axes([l-0.04, b, 0.03, h])
    c1 = plt.colorbar(cntrl1, cax=cb1, orientation='vertical')
    c1.set_label(r'$T$', fontsize=TickLabel, labelpad=5)

    plt.savefig(name, bbox_inches="tight")
    plt.close()
```

Visualization Process

1. For each timestep:
   - Call `getData-generic-heatEq` to extract data
   - Parse the output into NumPy arrays
   - Create a plot using matplotlib
   - Add appropriate colorbars and labels
   - Save as an image file
2. Parallelization:
   - Uses Python's multiprocessing to handle multiple timesteps simultaneously
   - Significantly speeds up the visualization process for large simulations

6.3 Running the Post-Processing Pipeline

To visualize your simulation results, follow these steps:

1. Compile the data extraction utility:

```
basilisk1d getData-generic-heatEq.c -o getData-generic-heatEq
```

2. Run the visualization script with appropriate parameters:

```
python Video-generic-heatEq.py --CPUs=4 --nGFS=100 --GridsPerR=128 \
  --ZMAX=4.0 --RMAX=4.0 --ZMIN=-4.0 --RMIN=-4.0 --tsnap=1.0 \
  --caseToProcess=1-conduction-2D-annulus --folderToSave=Video
```

3. Create a video from the generated images (optional):

```
ffmpeg -framerate 10 -pattern_type glob -i 'Video/*.png' -c:v libx264 -pix_fmt yuv420p
↪  video.mp4
```

Customizing Visualizations You can adjust the visualization parameters to highlight different aspects of your results:

- Change the colormap using the `cmap` parameter in `ax.imshow()`
- Adjust the value range with `vmin` and `vmax`
- Modify the plot layout or add additional subplots
- Include contour lines with `ax.contour()`

Conclusion

Throughout this tutorial, we've progressed from simple 1D steady-state heat conduction problems in vanilla C to complex geometries and dimensions using Basilisk C. The key takeaways include:

1. Basilisk significantly reduces the complexity and verbosity of numerical simulation code
2. The event-based programming model provides a clean separation of concerns
3. Basilisk's domain-specific language makes numerical methods more intuitive

4. Built-in modules like `diffusion.h` and `embed.h` provide powerful, optimized solvers
5. Extending to higher dimensions and complex geometries is straightforward
6. Post-processing tools are essential for analyzing and interpreting simulation results

Next Steps

- Explore other physical phenomena like advection-diffusion
- Implement variable diffusion coefficients (e.g., temperature-dependent)
- Combine with Navier-Stokes solvers for natural convection problems
- Add source terms for heat generation
- Experiment with more complex geometries using the embedded boundary method
- Create interactive visualizations using tools like Plotly or ParaView

References and Resources

1. Basilisk Documentation
2. Embedded Boundaries in Basilisk
3. Diffusion Module Documentation
4. Axisymmetric Coordinates
5. Adaptive Mesh Refinement
6. Basilisk View for Visualization

Need Help? If you encounter issues with any of the exercises, please check:

- Code syntax and structure
- Boundary condition implementation
- Event scheduling and dependencies
- Numerical stability considerations

Feel free to open an issue on the GitHub repository if you need further assistance.

Info

Thanks for reading, and feel free to dive deeper (or suggest improvements) via the GitHub repository!

Metadata

Author:: Vatsal Sanjay

Date published:: Jan 21, 2025

Date modified:: Jan 25, 2025 at 14:22

Back to main website

Home, Team, Research, Github